Verslag labo 5

Opdracht

Dit verslag zal gaan over dynamisch parallellisme en asynchrone operaties. Dit komt erop neer dat we twee programma's met elkaar gaan vergelijken. Voor de vergelijking gaan we vier kernels doorlopen waarbij er op een voorgedefinieerde array telkens de minimale waarde, de maximale waarde, de som van alle getallen in de array en het product wordt gegeven. Bij het eerste programma maken we gebruik van synchrone operaties waarbij we de kernels achter elkaar laten werken. Hierbij zal de volgende kernel pas werken als de vorige gedaan heeft. In het tweede programma zullen we gebruik maken van asynchrone programma's en parallellisme waarbij de kernels niet hoeven te wachten op elkaar. De volgende kernel kan dus meteen starten zonder te kijken of de vorige gedaan heeft. Hiervoor bekijken we het timing-aspect tussen beide programma's en de invloed van de grootte van de array.

Werkwijze

Als werkwijze is deze best voordehandliggend. We zijn begonnen van de code uit labo 2 waarbij we al een max-functie hebben gecreëerd m.b.v. reductie. Hier hebben we deze verder uitgebreid zodat de kernels van min, som en vermenigvuldiging ook werkten. Pas dan hebben we het tweede programma geschreven voor de asynchrone werking waarbij we kernel per kernel toevoegen en aanpassen om de werking te testen.

Code

Synchrone werking

```
__global__ void reductionMin(int *input, int *output, int size) {
    unsigned int tid = threadIdx.x;
    __syncthreads();

    for (int s = size / 2; s > 0; s >>= 1) {
        if (tid < s) {
            input[tid] = min(input[tid], input[tid + s]);
        }
        __syncthreads();
    }
    *output = input[0];
}
```

In de kernel gaat men na de tid te bepalen een for-loop aanmaken die begint op waarde size/2 en per keer zijn waarde deelt door 2. Dit zal ervoor zorgen dat in een array met size 16 deze per beurt wordt gereduceerd naar 8, 4, 2 en tenslotte 1. Vervolgens gaat men enkel de tid's doorlopen die nodig zijn m.b.v. een if-loop van tid $<$ s. Dit deel code is voor alle kernels hetzelfde waarbij enkel het gedeelte hier binnen deze if-loop wordt aangepast. Voor de minimale waarde vergelijkt men op de plaats van tid de input van tid en tid + s. Dit komt overeen dat 0 en 8 worden vergeleken, 1 en 9,.. voor de eerste loop. Deze worden dan weggeschreven op de tid zelf waardoor enkel nog de eerste acht een nuttige waarde bevatten en we de lus kunnen delen door 2. Hierbij is het dus zeer belangrijk dat de for-loop enkel en alleen de gehele array meeneemt waarbij er anders fouten gaan ontstaan en deze enkel een nulwaarde geeft op de uitgang d.m.v. de waarden groter dan de array-size.

Asynchrone werking en parallellisme

In dit programma wordt er gebruik gemaakt van cudaMemcpyAsync waarbij er nog een parameter stream wordt aan toegevoegd. Deze stream bepaalt de plaats waar de memcpy zal worden uitgevoerd terwijl de CPU verder kan gaan met zijn instructies. Verder worden de kernels asynchroon gemaakt wat ervoor zorgt dat er meerdere kernels tegelijk kunnen runnen zonder dat deze invloed hebben op elkaar qua timing. De cudaStreamSynchronize zorgt ervoor dat alle instructies binnen deze stream zijn uitgevoerd alvorens verder te gaan met de code.

Conclusie

Wanneer we op de CPU niet moeten wachten op het eindigen van een proces dat op de GPU bezig is, kunnen we vanzelfsprekend verwachten dat dit in de meeste gevallen een voordeel zal geven op vlak van de berekentijd van het programma. Ook bij ons zien we een daling in berekentijd, zoals we kunnen zien op onderstaande grafiek. Hierbij zien we op de horizontale as de grootte van onze array waar we de verschillende berekeningen op toepassen, en op de verticale as de benodigde tijd hiervoor. Vooral op de eerste waarde en op het einde van de horizontale as zien we een drastisch verschil tussen synchroon en asynchroon werken.

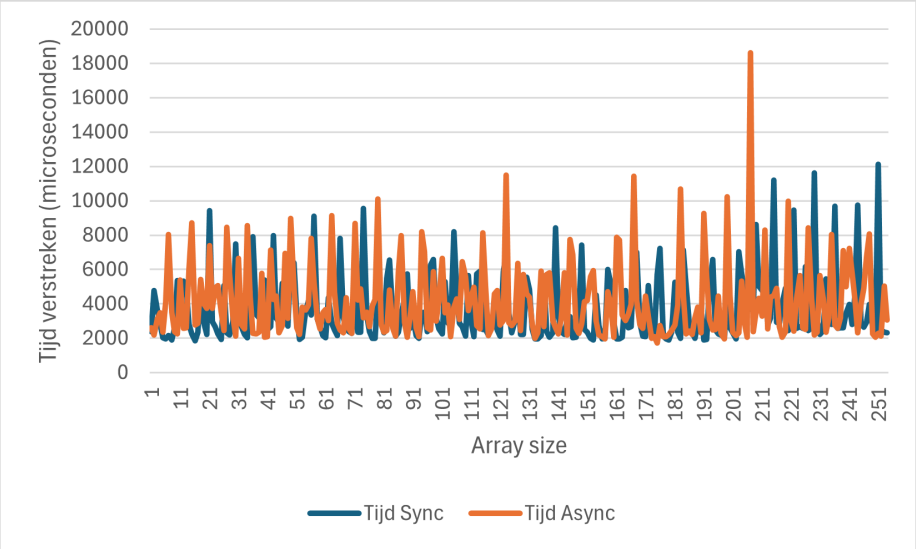


Figure 1: Synchronous vs asynchronous processing